SET 2430 ⊡ Assignment 3: Summations & Big-O

SET 2430 Assignment 3

Loop Summations and Asymptotic Runtime Analysis

Part A. Nested Triangular Loop Analysis

Consider the following mystery function:

```
def mystery(n):
    count = 0
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            count += 1
    return count
```

Requirements:

1. Write the formal summation expression (Sigma notation):
   Sum_{i=1 to n} (i) = 1 + 2 + 3 + ... + n
2. Derive the closed-form algebraic formula:
   n * (n + 1) / 2 = (n^2 + n) / 2
3. State the Big-O and Big-Theta time complexity:
   Theta(n^2) and O(n^2)

Part B. Running Sum of 1D Array

Given an array nums:

```
def runningSum(nums):
    res = []
    current = 0
    for x in nums:
        current += x
        res.append(current)
    return res
```

Requirements:

1. Implement the algorithm in Python.
2. Express the total runtime mathematically:
   Sum_{i=1 to n} (c) = c * n
3. State the Big-O time and space complexity:
   Time Complexity: O(n)
   Auxiliary Space Complexity: O(n) for result array